

Make the architecture decision reviewable

I propose one shared record behind the paper, slides and diagrams.

Evidence

Show what is known, what is inferred and what remains uncertain.

Decision

State the choice, its strongest rival and what would reverse it.

Story

Explain the same recommendation in two useful forms.

Polish alone does not explain why to choose

The failure I target is lost reasoning and drift between artifacts.

A claim changes

The source no longer supports a pivotal capability.

A decision depends on it

The chosen design may now require a different mechanism.

Both outputs must change

The paper and presentation need the same correction.

Start with invariants, then derive the process

I ask what must remain true before I select a tool or pattern.

- Every factual claim has inspectable support.
- Every pivotal choice has a credible alternative and reversal trigger.
- Every diagram and slide preserves the same architecture and conditions.
- Changed canonical content requires another review.

A structured loop must earn its extra work

For a small task, a simple checklist may be the better design.

One prompt / checklist

Low setup cost. A strong baseline for occasional short work.

File-based loop

Durable evidence and repeated checks. More records to maintain.

Managed service

Consider when concurrency or independent approval demands it.

Try to overturn the recommendation

I define what would change my mind before declaring a winner.

Value challenge

If defects do not improve and effort rises, prefer the checklist.

Authority challenge

If approval must be independent, local writable files are insufficient.

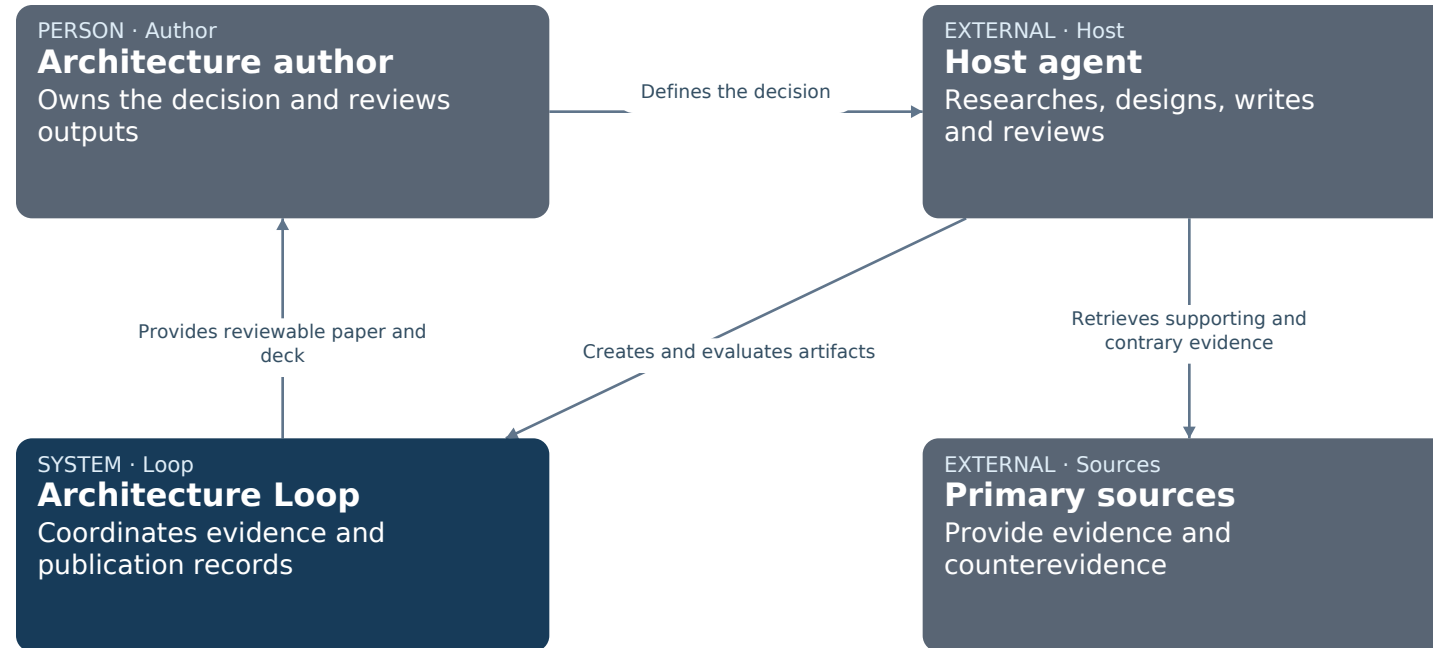
Portability challenge

If the selected host cannot resume and export, fix the adapter or narrow the claim.

The host does the thinking; the kit keeps records

C1 • System context

Scope: Architecture Loop • Loop



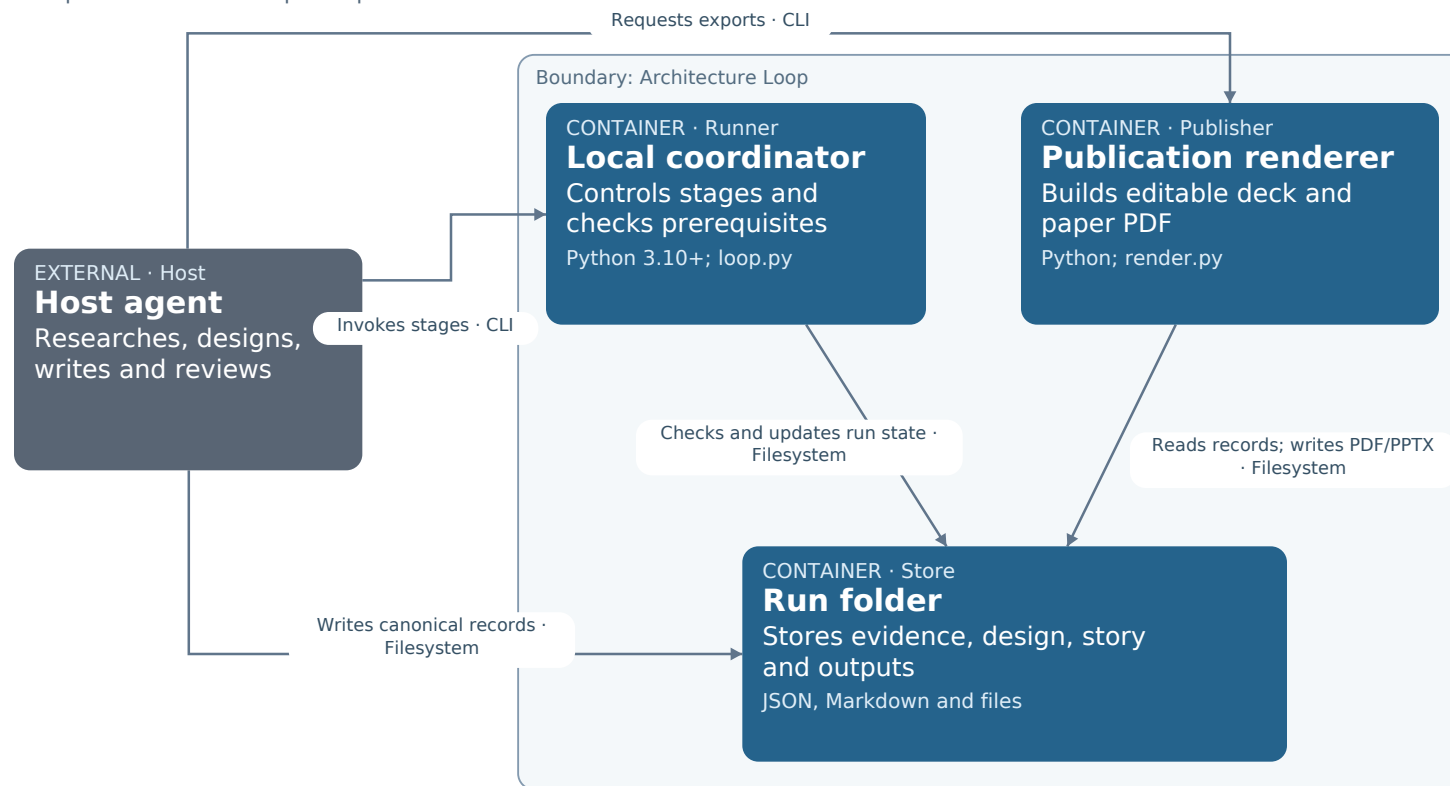
Person = accountable author. Dark blue = kit. Gray = external systems. Arrows describe directed interactions.

C1 makes the boundary and the author's accountability explicit.

Three local containers keep the kit portable

C2 · Containers within Architecture Loop

Scope: Architecture Loop · Loop



Outlined boundary = kit ownership. Blue = application or data-store container. Gray = external host. Arrow labels include protocol.

C2 separates the coordinator, shared folder and optional renderer.

One bounded loop carries the work to export

Research and critique can reopen the earliest affected stage.

Discover and decide

Frame → research → challenge. Change the hypothesis when evidence requires it.

Design and explain

Design → story → draft.
Derive both forms from the shared records.

Review and deliver

Review → export. Reopen earlier work when meaning changes.

A new claim invalidates the old review

A fingerprint binds a review to exact files, not to their truth.

Detect

Cumulative checks identify missing references and stale review bytes.

Repair

Snapshot, reopen research and update the dependent decision and outputs.

Reassess

Review current content, render again and inspect the new files.

One story can serve two reading modes

The deck simplifies detail while preserving the paper's conditions.

Paper

Full context, evidence, mechanisms, alternatives and validation.

Deck

One conclusion per slide, clear visuals and readable source notes.

Shared contract

Same names, numbers, caveats, diagrams and decision ask.

Integrate through files and explicit instructions

Portability depends on the capabilities available in the host.

Claude Code

Merge the small adapter into CLAUDE.md; explicitly load the kit.

GitHub Copilot

Merge repository instructions; verify that the selected host loads them.

Other environments

Use the master prompt and the same durable run records.

Measure whether the process improves decisions

I would compare the loop with a checklist on representative briefs.

Decision quality

Count unsupported pivotal claims, missed alternatives and contradictions.

Reader understanding

Can a newcomer explain the mechanism, tradeoff and reversal condition?

Total effort

Track author time, reviewer time, model cost and meaningful rework.

My recommendation is a bounded pilot

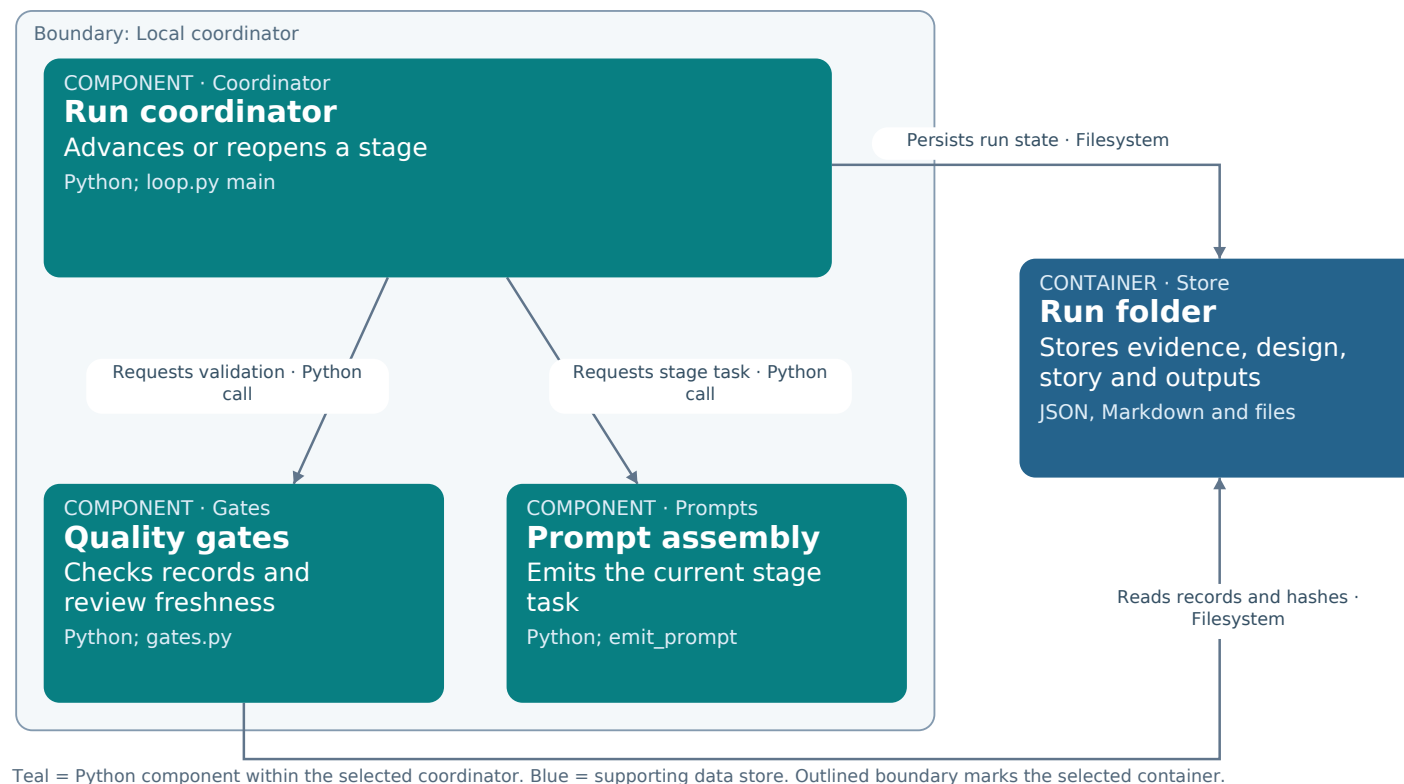
Adopt broadly only if the improvement justifies the added work.

- Start with one representative architecture brief.
- Define acceptable defects, reader understanding and effort in advance.
- Use the pilot to decide whether the extra records and checks earn their cost.
- Add managed coordination only when a measured constraint requires it.

The coordinator separates transitions from checks

C3 • Components inside the local coordinator

Scope: Local coordinator • Runner

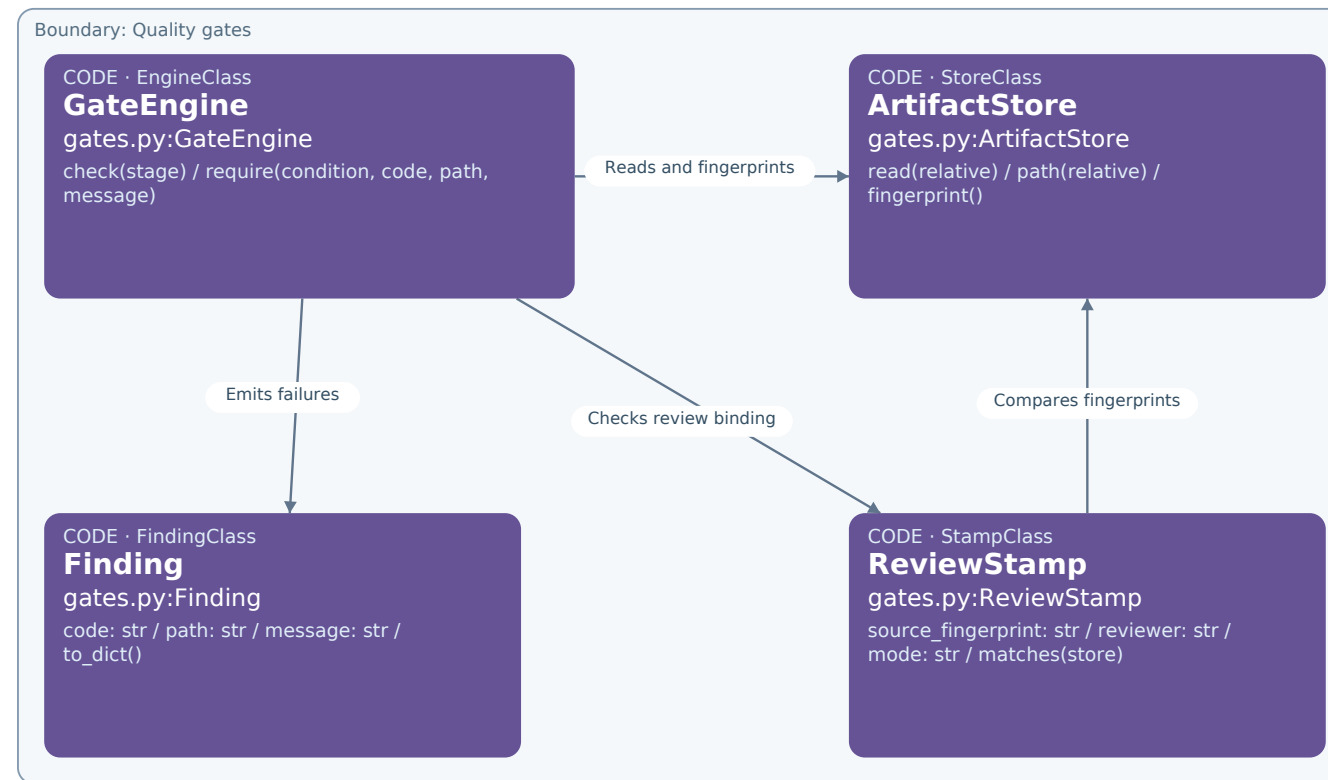


C3 zooms into the runner; the folder remains a supporting container.

Review freshness is implemented in four classes

C4 • Code within quality gates

Scope: Quality gates · Gates



Purple = implemented Python class in gates.py. Members show selected methods or fields. Arrows show directed code dependencies.

C4 shows selected real code in gates.py, not a deployment view.